LAB EXERCISE - 1

Installing Python, Setting up VS Code / PyCharm, and Jupyter Notebook — macOS Guide

Name	Sachin Shinde		
USN	25BTECH099	**Branch**	CSE - AIDE "B"

Before we can start writing and running Python programs, we need three things in place: Python itself (the language and its interpreter), a code editor or IDE to write programs in (VS Code and/or PyCharm), and Jupyter Notebook for interactive, cell-by-cell coding. This guide walks through setting up all three on macOS, from a completely fresh machine to a working environment where you can run your first program.

Contents

1. Installing Python on macOS
2. Setting up Visual Studio Code (VS Code)
3. Setting up PyCharm
4. Setting up Jupyter Notebook
5. Verifying Your Setup

1 Installing Python on macOS

macOS actually ships with a system version of Python, but it's usually outdated and Apple doesn't want you touching it for your own projects — messing with the system Python can break tools macOS relies on internally. So the very first step is installing a clean, independent copy of Python that belongs to you. The easiest and most reliable way to do this is by downloading the official installer from python.org.

Step 1.1 — Download the installer

Open your browser and go to **python.org/downloads**. The site automatically detects that you're on a Mac and shows a big yellow button to download the latest stable release (for example, Python 3.12 or 3.13). Click it — a **.pkg** file will start downloading.

Step 1.2 — Run the installer

Once the download finishes, open the **.pkg** file from your Downloads folder. This launches the standard macOS installer wizard. Click *Continue* through the introduction, read-me, and license screens (you can just accept the license), then click *Install*. macOS will ask for your account password since it's installing software system-wide — enter it and let the installer finish.

Step 1.3 — Let the extra scripts run

Towards the end, a Finder window usually pops up showing a few extra files, including *Install Certificates.command*. Double-click that file and let it run — it installs the SSL root certificates Python needs to make secure internet connections (without it, things like **pip install** can fail later with certificate errors).

Step 1.4 — Confirm it's installed

Open the **Terminal** app (press Cmd+Space, type "Terminal", hit Enter) and run the following commands to confirm Python and pip (Python's package manager) are ready to go:

```
python3 --version
pip3 --version
```

You should see version numbers printed back, something like `Python 3.12.4` and a matching pip version. On macOS you'll almost always type **python3** and **pip3** rather than plain **python** and **pip**, since the plain names are reserved for the old system Python 2 installation.

> **TIP**
> If you plan to work on multiple projects with different Python versions later, look into installing Python via **Homebrew** (`brew install python`) or a version manager like **pyenv**. For a single lab setup like this one, the official installer above is perfectly fine and the simplest route.

2 Setting up Visual Studio Code (VS Code)

VS Code is a free, lightweight code editor from Microsoft that has become the default choice for a lot of Python developers because it's fast, highly customizable through extensions, and works well for everything from small scripts to large projects.

Step 2.1 — Download VS Code

Go to **code.visualstudio.com** and click the *Download for macOS* button. This downloads a **.zip** file containing the application.

Step 2.2 — Install it

Open the downloaded zip (double-clicking it should extract **Visual Studio Code.app** automatically). Drag that app into your **Applications** folder. That's the entire installation — no wizard, no password prompt needed.

Step 2.3 — Enable the 'code' command in Terminal

This step is optional but genuinely useful: it lets you type **code .** in Terminal to open the current folder in VS Code. Open VS Code, press **Cmd+Shift+P** to open the Command Palette, type "Shell Command", and select *Shell Command: Install 'code' command in PATH*.

Step 2.4 — Install the Python extension

Click the Extensions icon in the left sidebar (it looks like four small squares), search for **Python**, and install the official extension published by Microsoft. This single extension gives you syntax highlighting, IntelliSense (autocomplete), debugging support, and the ability to select which Python interpreter to run your code with.

Step 2.5 — Select your Python interpreter

Open any **.py** file (or create a new one and save it with a .py extension). In the bottom-right corner of the VS Code window you'll see the interpreter version — click it and choose the Python 3 installation you set up in Section 1.

Step 2.6 — Run your first script

Create a new file called **hello.py**, type the line below, save it, and click the Run/Play triangle in the top-right corner (or right-click in the editor and choose *Run Python File in Terminal*).

```
print("Hello, world!")
```

You should see `Hello, world!` printed in the integrated terminal panel that opens at the bottom of the window.

3 Setting up PyCharm

PyCharm is a full-featured IDE built specifically for Python by JetBrains. Compared to VS Code, it's heavier but comes with a lot of Python-specific tooling built in out of the box (refactoring tools, a powerful debugger, database tools, and deep project management). The free **Community Edition** is more than enough for coursework and lab exercises.

Step 3.1 — Download PyCharm Community Edition

Visit **jetbrains.com/pycharm/download**, make sure the macOS tab is selected, and choose your chip type — *Apple Silicon* for M1/M2/M3/M4 Macs, or *Intel* for older Macs. Under the free **Community** column, click *Download*.

Step 3.2 — Install the app

Open the downloaded **.dmg** file. A window will appear showing the PyCharm icon and a shortcut to your Applications folder — drag the PyCharm icon onto the Applications folder icon to install it, then eject the disk image.

Step 3.3 — First launch and setup wizard

Open PyCharm from Applications (macOS may warn you it was downloaded from the internet — click *Open* to allow it). On first launch you'll be guided through a short setup wizard where you can pick a UI theme (Light/Dark) and keymap — the defaults are fine to start with.

Step 3.4 — Create a new project

From the Welcome screen, click **New Project**. Choose a location/folder for it, and under *Python Interpreter* select **New environment using Virtualenv** (recommended — this keeps each project's packages isolated) or point it at the system interpreter you installed in Section 1. Click **Create**.

Step 3.5 — Run your first script

Right-click the project folder in the left sidebar, choose *New* > *Python File*, name it **hello.py**, and add the same print statement as before:

```
print("Hello from PyCharm!")
```

Right-click anywhere inside the editor and select *Run 'hello'* (or click the green play button next to the line, or press **Ctrl+Shift+R**). The output appears in the Run panel at the bottom.

> **NOTE**
> You don't need both VS Code and PyCharm — pick whichever fits your workflow. Many students keep VS Code for quick scripts and general-purpose editing, and switch to PyCharm for larger, structured projects. Both are used side by side in this guide simply so you're comfortable with either one.

4 Setting up Jupyter Notebook

Jupyter Notebook lets you write and run Python in individual "cells" inside a web browser, with the output (including plots and tables) displayed right below the code that produced it. It's especially popular for data analysis, experimentation, and lab work where you want to see results immediately without re-running an entire script.

Step 4.1 — Open Terminal

Press **Cmd+Space**, type "Terminal", and press Enter. All the Jupyter setup happens through the command line.

Step 4.2 — Install Jupyter with pip

With Python already installed from Section 1, installing Jupyter is a single command:

```
pip3 install notebook
```

This downloads and installs Jupyter along with its dependencies. It may take a minute or two depending on your internet connection.

Step 4.3 — Launch Jupyter Notebook

Navigate to the folder where you want your notebooks to live (for example your Desktop or a dedicated project folder), then start the notebook server:

```
cd ~/Desktop
jupyter notebook
```

This starts a local server and automatically opens a new tab in your default browser showing the Jupyter file dashboard. Keep the Terminal window open in the background — closing it shuts the server down.

Step 4.4 — Create a new notebook

In the browser tab, click the **New** button on the right and select **Python 3** (sometimes labelled *Python 3 (ipykernel)*). This opens a fresh notebook with one empty, ready-to-use cell.

Step 4.5 — Run your first cell

Click into the first cell, type the following, then press **Shift+Enter** to run it (Shift+Enter runs the current cell and moves to the next one):

```
print("Hello from Jupyter Notebook!")
```

The output appears directly beneath the cell. You can keep adding new cells (with the **+** button in the toolbar) to build up your work step by step, mixing code cells with Markdown cells for notes and explanations.

> **WARNING**
> If running `jupyter notebook` gives a "command not found" error, it usually means pip installed it to a location that isn't in your Terminal's PATH. Try `python3 -m notebook` instead — this always works because it runs Jupyter through Python directly.

5 Verifying Your Setup

Before moving on to actual lab work, it's worth doing a final sanity check to make sure everything from this guide is working correctly.

Check	Command / Action	Expected Result
Python installed	`python3 --version`	Prints a version like Python 3.12.x
pip installed	`pip3 --version`	Prints a matching pip version
VS Code runs Python	`Run hello.py via the Play button`	Output appears in integrated terminal
PyCharm runs Python	`Run hello.py via right-click > Run`	Output appears in Run panel
Jupyter works	`jupyter notebook, then run a cell`	Browser opens; cell output shows below it

Once all five checks pass, your Mac is fully set up for Python development — you have a working interpreter, two capable editors/IDEs to choose from, and an interactive notebook environment ready for the lab exercises ahead.

Prepared for Lab Exercise 1 by Sachin Shinde · USN 25BTECH099, Branch CSE - AIDE "B".